

Упражнение 2 – Системи за управление на версиите

Цел на упражнението: Основната цел на второто упражнение е запознаване на студентите с идеята за необходимост от следене на софтуера. На вниманието на студентите ще бъде представен исторически преглед за развитието на идеята за следене на версиите. Разглеждат се основните модели, на които са базирани съвременните системи за следене на версии. Анализират се и се подлагат на дискусия проблемите, свързани със заключване на файлове и с обединяване на версиите на файлове. По време на текущото и следващото упражнение студентите ще имат възможност да се запознаят с използването на системите за следене на версии: SVN, Git и TFS.

Ключови думи: система за следене на версиите; version control system; исторически преглед; основни понятия; Lock-Modify-Unlock Model; Copy-Modify-Merge Model; SVN, Git, TFS.

Съдържание

1. Система за контрол на версиите (VCS).....	1
2. Исторически преглед.....	2
3. Основни понятия[1].....	3
3.1.Понятия, свързани с централизираните VCS системи.....	3
3.2.Понятия, свързани с разпределените VCS системи.....	5
4. Предимства на разпределените VCS системи[1].....	7
5. Недостатъци на разпределените VCS системи[1].....	8
6. Модели, на базата на които работят системите за следене на версии[2].....	9
6.1.Lock-Modify-Unlock Model.....	9
6.2.Copy-Modify-Merge Model.....	10
7. Добри практики[1].....	12
8. Subversion (SVN).....	13
9. Git[3].....	13
10. Team Foundation Server (TFS) [4].....	13
11. Задачи за изпълнение.....	13
Задача 1.....	13
Задача 2.....	13
Задача 3.....	14

1. Система за контрол на версиите (VCS)

Системата за контрол на версиите представлява софтуер, който помага на разработчиците от един екип да работят заедно като им позволява да пазят пълна история на своята работа.

Основни цели на VCS системите са [1]:

- предоставяне на възможност за паралелна работа по конкретен проект;
- предоставяне на възможности за разрешаване на конфликти, настъпващи при паралелната работа на разработчиците;
- предоставяне на възможност за съхранение на всяка версия на всичко, което някога е било част от проекта.

Обикновено VCS системите се използват по време на разработката на софтуерни проекти като пазят историческа информация за всеки от файловете на проекта. Състоянието на всеки файл се определя с последователно инкрементиращи се числа – номер на версията – напр. „1.0“, „1.1“ и т.н. Всяко състояние на файла се асоциира с конкретен потребител на VCS системата, който е извършил операция

по съхраняване на състоянието.

2. Исторически преглед

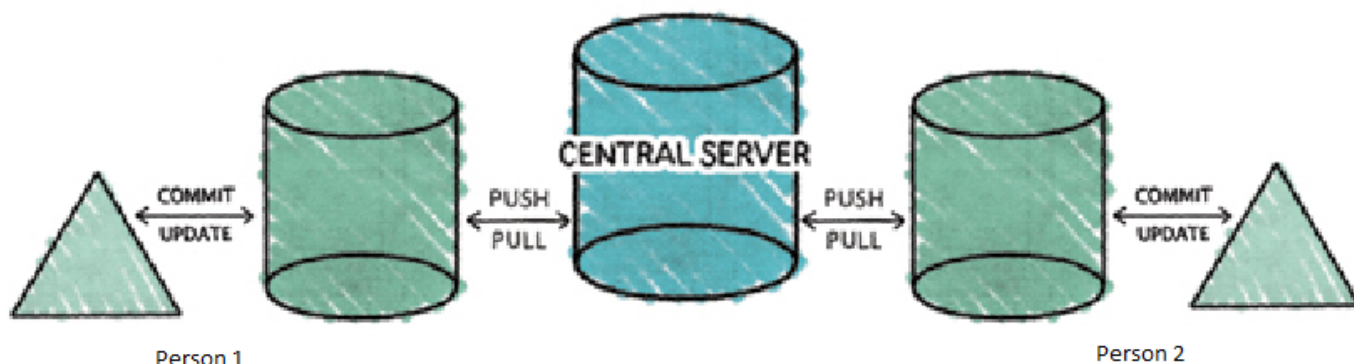
В исторически план се разглеждат три поколения системи за следене на версии [1]:

- В **първото поколение** системите са извършвали конкурентната работа с ресурси единствено със заключвания. Това означава, че само един човек е можел да работи с един файл за определен момент от време. Най-общо конкурентният достъп може да се опише като „lock“. Такива VCS системи са: RCS, SCCS;
- **Второто поколение** системи дават възможност за едновременна работа на няколко човека по един и същи файл в едно и също време, но преди да се запази ново състояние в хранилището, потребителите трябва да обединят своите версии. Най-общо конкурентният достъп може да се опише като „merge before commit“. Тези VCS системи могат да се нарекат **централизирани VCS (centralized)**. Такива VCS системи са: CVS, SVN, TFS. Обобщена схема на използването на централизирана VCS има на Фигура 1;
- **Третото поколение** системи позволяват обединяването на версии и записването на състояние в хранилището да бъдат отделни процеси. Най-общо конкурентният достъп може да се опише като „commit before merge“. Тези VCS системи могат да се нарекат **разпределени VCS (distributed/decentralized)**. Такива VCS системи са: Bazaar, Git, Mercurial. Обобщена схема на използването на разпределена VCS има на Фигура 2.

Голяма част от професионалните разработчици използват VCS системи от второ поколение, но броят на хората, използващи VCS от трето поколение, нараства.



Фигура 1: Обобщена схема за използването на централизирана система за следене на версиите



Фигура 2: Обобщена схема за използването на разпределена система за следене на версиите

3. Основни понятия[1]

Забележка: За яснота се приема, че описаните по-долу команди се отнасят за цялостен проект, който се следи от VCS системата.

3.1. Понятия, свързани с централизираните VCS системи

- **Хранилище (Repository)** – Хранилището е мястото, където се съхранява работата на разработчиците – всички проекти, по които разработчиците работят, и файлове и директории, от които те се състоят. Основната разлика между хранилище и обикновена файлова система е, че хранилището пази история. Всяка промяна в хранилището, включително и изтриването на някакъв файл от проект, увеличава обема на хранилището, защото абсолютно всяка промяна се съхранява;
- **Работно копие (на проект)** – За да може разработчикът да използва и да работи върху даден проект, необходимите файлове от хранилището трябва да се съхранят и локално на машината на разработчика. Като използва локално работно копие на проекта, разработчикът може да работи паралелно с другите участници в екипа, без да се променят оригиналните файлове от работното копие, докато това не се наложи. На практика работното копие е мястото, където разработчикът може да прави промени.

Забележка: Работното копие не е огледално копие на проекта от хранилището. Към всеки от файловете и директориите има допълнителни метаданни. Чрез тях VCS системата извършва следенето на промените по ресурсите на проекта;

- **checkout (на проект)** – Команда за създаване на работно копие. Това е действие, което се прилага когато разработчик иска да вземе на локалната си машина актуалното състояние на конкретен проект от хранилището, за да започне да работи по него;
- **commit (на промяна към проект)** – Команда за прилагане на модификациите по работното копие в хранилището, т.е. в хранилището се отразяват множеството промени, направени по работното копие. Commit операцията е тази, която в действителност модифицира данните в хранилището. Някои VCS системи дефинират понятие *pending changeset* множество от промени, които все още не са приложени към хранилището. Командата *commit* прилага промените от *pending changeset*-а върху хранилището;
- **update (на файл, директория или проект)** – Команда за обновяване на работното копие според данните в хранилището. Тази команда взема най-актуалното състояние на файловете от хранилището и обновява файловете в работното копие;

- **add** – Команда за добавяне на файл или директория към хранилището. Тази операция се извършва когато имаш файл или директория в работното копие, за които VCS системата няма информация, тъй като до сега не са били част от хранилището;
- **delete** – Команда за изтриване на файл или директория от хранилището. Обикновено изтриването се извършва веднага от работното копие на проекта, но реалното изтриване в хранилището се отбелязва едва след операция *commit*. Всъщност никога не се извършва реално изтриване на информация от хранилището. Просто файлът или директорията, които са били изтрити от работното копие, се отбелязват за изтрити. Ако в последствие се извърши изтегляне на актуалното състояние на проекта от хранилището (операция *checkout*), в новото работно копие изтриването ще бъде отразено;
- **rename** – Команда за промяна на името на файл или директория. Обикновено преименуването се извършва веднага в работното копие на проекта и се отразява в *pending changeset*-а. От друга страна в зависимост от конкретната VCS система се прилага различна логика при пренасянето на промяната в хранилището;
- **move** – Команда за промяна на местоположението на файл или директория. Обикновено преименуването се извършва веднага в работното копие на проекта и се отразява в *pending changeset*-а. От друга страна в зависимост от конкретната VCS система се прилага различна логика при пренасянето на промяната в хранилището;
- **status** – Команда за получаване на всички промени, направени по работното копие на проекта. Когато се правят промени по работното копие, те се отбелязват в *pending changeset*. Командата *status* показва какви са промените, които ще бъдат приложени върху данните в хранилището при изпълнение на команда *commit*;
- **diff** – Команда за показване на детайлите по промените, направени по работното копие. Разликата е между командите *diff* и *status* е, че командата *status* показва всички промени, но без подробна информация за тях. Ако е необходимо да се види какви точно промени са направени по съдържанието на даден файл, трябва да се използва командата *diff*;
- **revert/undo (на промени)** – Команда за връщане/отмяна на промените, направени по работното копие. Пример за използването на тази команда: На локалната машина на разработчика има актуално работно копие на проекта. Разработчикът решава да разработи нова функционалност към проекта. След известно време на разработка, възложителят заявява, че няма нужда от въпросната функционалност. В този случай, разработчикът, използвайки командата *revert*, може да върне състоянието на работното копие към момента преди започването на разработката на нежеланата функционалност;
- **log** – Команда за показване на история на промените в хранилището. Всяко хранилище следи всяка версия на всеки файл, който някога е съществувал в него. Операцията *log* е начинът да се проследят тези версии. Всеки лог съдържа допълнителна информация за промяната като:
 - потребител, който е направил промяната;
 - момент на извършване на промяната;
 - коментар, който потребителят е сложил при извършването на промяната.
- **tag (на проект)** – Команда за даване на значещо име на конкретна версия на проект от хранилището. Когато по даден проект има завършена част, то е хубаво да се запази състоянието на проекта. Тогава на състоянието му се дава значещо име, така че при необходимост проектът да може да се върне към състоянието си от запазения момент;
- **branch (на проект)** – Команда за създаване на нова линия (клон) на разработка. Операцията *branch* се изпълнява когато разработката на даден проект трябва да се раздели в две различни насоки.

Пример 1: Даден проект е разработен до някаква степен. Предстои разработването на нова функционалност, но екипът от разработчици не може да вземе решение каква технология да използва за разработката на новата функционалност. В този случай проектът може да се клонира и паралелно да се разработва една и съща функционалност по различни технологии.

Пример 2: Даден е проект с версия 3.0. Удобно е да се направи клониране на проекта, за да може разработката на следващата версия (4.0) да продължи, независимо от версиите за корекции на бъгове по версия 3.0 (3.0.x);

- **merge (на клонове)** – Команда за автоматично сливане на клонове на проект. Обикновено когато е била приложена технологията на разделяне на разработката на проект в различни клонове, то в последствие се появява необходимостта от обединяване/сливане на тези клонове;
- **resolve** – Прихващане на конфликти, при автоматизираното извършване на команда *merge*. В някои случаи сливането на клонове може да се направи напълно автоматичното. В някои други случаи, обаче, е необходима намесата на човек. Пример, в който е нужна намесата на човек за решаване на възникнал конфликт: Даден файл съществува в единия от сливаните клонове, но в другия клон файлът липсва. Тази ситуация не може да бъде решена от автоматичния *merge*. Нужно е човек да анализира ситуацията и да вземе решение дали файлът трябва да съществува или не като резултат от сливането на клоновете;
- **lock** – Операция по заключване на файл. По този начин не се позволява на разработчиците да работят по файл, който е в процес на модифициране от друг разработчик. Тази команда не се предоставя от всички VCS системи и наличието ѝ се определя от модела, на базата на който работи съответната VCS система. Повече информация за заключването има в т. 6.1 Lock-Modify-Unlock Model, стр. 9.

3.2. Понятия, свързани с разпределените VCS системи

- **clone** – създаване на нова инстанция на хранилището, което представлява копие на друго хранилище. Основната разлика между централизираните и разпределените VCS системи е понятието „*инстанция на хранилището (repository instance)*“. При разпределените VCS системи хранилище се създава еднократно в началото, а всеки следващ път се създава нова инстанция на хранилището чрез командата *clone*. При централизираните VCS системи хранилището съществува само на едно място – на централен сървър. От друга страна при разпределените VCS системи, хранилището съществува на повече от едно места – разполага се с множество инстанции на хранилището. Наличието на множеството хранилища изисква необходимост от синхронизиране между тях. Повечето операции при разпределените VCS системи се случват в локалната инстанция на хранилището. Единственият момент, в който е нужна мрежова комуникация, е когато отделните инстанции се синхронизират;
- **push** – Копиране на промените от локалното хранилище към отдалечено хранилище. Тази операция се използва за синхронизиране между две инстанции на хранилища. Тази операция е иницирирана от локална инстанция на хранилището, която иска да запише своите промени към отдалечена инстанция. Обикновено инстанцията, към която се записват промените, е инстанцията, от която е била създадена инстанцията, иницирираща *push* операцията;
- **pull** – Копиране на промените от отдалечена инстанция на хранилище към локалното хранилище. Тази операция също се използва за синхронизиране между две инстанции на хранилища. Операцията е иницирирана от локалната инстанция на хранилището, която иска да получи промените от отдалечена инстанция на хранилището. Обикновено инстанцията, от която се вземат промените, е инстанцията, от която е била създадена инстанцията, иницирираща *pull* командата;
- **Насочен ацикличен граф (дърво)** – Следенето на версиите при разпределените VCS е по-сложно отколкото при централизираните VCS системи. Сложността при разпределените системи произлиза от това, че трябва да се следят версии между инстанции на едно и също хранилище.

При централизираните VCS следенето на версиите може да се опише като линеен модел, показан на Фигура 3. Най-просто схемата на работа изглежда по следния начин:

- взема се последната версия, която е в хранилището;
- извършват се промени;
- промените се връщат в хранилището като се създава нова последна версия.



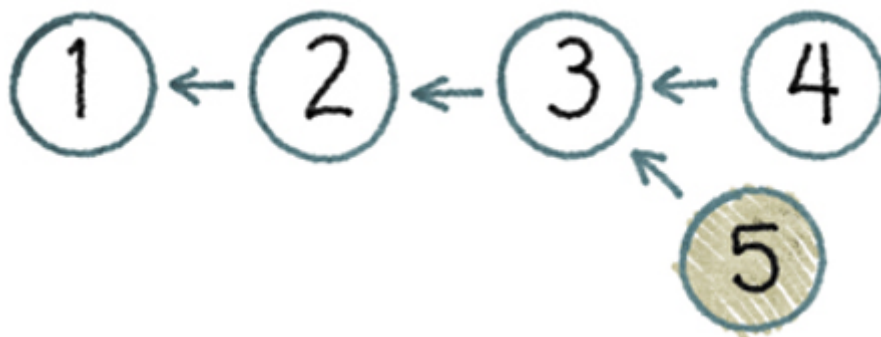
Фигура 3: Централизирана система за следене на версиите - Модел за следене на историята

Основно **предимство** на линейния модел за следене на историята на промените е, че във всеки момент може да се каже коя версия е последната.

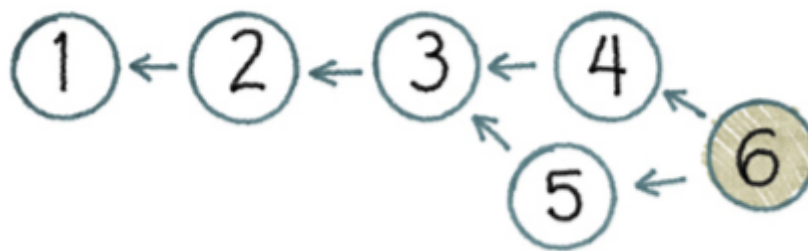
Основен **недостатък** на линейния модел е, че може да се направи commit на нова версия САМО ако версията е базирана на последната версия в хранилището. Проблемът се забелязва при следната схема на работа (която се случва доста често):

- взема се последната версия от хранилището (напр. версия 3);
- правят се промени по нея;
- докато се правят промените някой друг вече е променил версията в хранилището и там последната версия е друга (напр. версия 4);
- когато се направи опит да се прехвърлят промените в хранилището, то това е невъзможно, тъй като се оказва, че промените са правени по стара версия (версия 3), а не по последната версия (версия 4).

Проблемът е показан на Фигура 4 - Линейният модел за следене на история не позволява да се създаде версия 5. Той принуждава първо да се вземат разликите между версии 3 и 4, а след това върху получената версия 4 да се нанесат промените, нужни за създаването на версия 5.



Фигура 4: Централизирана система за следене на версиите - Проблем с модела за следене на историята



Фигура 5: Разпределена система за следене на версиите -
Модел за следене на историята

Насоченият ацикличен граф се прилага от децентрализираните VCS системи за следене на историята. Той решава проблема на линейния модел на централизираните VCS системи като добавя сложността да се указва за всяка версия, върху кои други версии е базирана. Пример е показан на Фигура 5.

Възможно е обаче насоченият ацикличен граф да се окаже много сложна структура. Това е основната причина разработчиците да предпочитат централизираните VCS системи.

Забележка: При децентрализираните системи също има еквивалент на *merge* – за обединяване на версии, но при децентрализираните системи операцията *merge* може да се случи на по-късен етап за разлика от централизираните системи, където *merge* се случва при *commit*.

4. Предимства на разпределените VCS системи[1]

- Личен workspace – Идеята за личния workspace е обща за всички VCS системи. При централизираните VCS системи разработчиците получават собствено място, където да работят върху работно копие. При разпределените VCS системи развиват идеята като дават на разработчика копие на цялото хранилище;
- Бързина – Обикновено разработчиците не осъзнават бързината на разпределените VCS системи. Тя произлиза от това, че разработчикът основно работи на своята локална машина и комуникацията с централния сървър е ограничена;
- Възможност за offline работа – Тази възможност произлиза от локалното хранилище, с което разработчикът разполага. Във всеки момент той може да използва локалното си хранилище, за да съхрани състоянието на работата си. От друга страна, ако разработчикът използва централизирана VCS система, той задължително трябва да установи връзка с централизирания сървър, за да съхрани състоянието на своя проект;
- Възможност за географско разпределение на работата – Ако има два екипа разработчици, работещи по един и същи проект, които са разположени физически в две различни местности, то те трябва да синхронизират работата си. Ако те използват централизирана VCS система, то трябва да се вземе решение в кой от двата офиса ще се намира централизираното хранилище. Ако за място на централизирания сървър се избере офиса на единия екип, то комуникацията на другия екип със сървъра ще минава през Интернет и ще бъде изключително забавена. Децентрализираните VCS системи решават този проблем като дават възможност и двата екипа да си имат централизирани сървъри, които периодично се синхронизират помежду си с *push* и *pull* операции;
- Възможност да се създават различни инстанции на хранилището с различни цели – според нуждите на екипа за реализирането на конкретния проект;
- По-лесно сливане на версии – По правило операцията по разделяне на клонове (*branching*) е по-лесна за реализиране от операцията по сливане на клонове (*merging*). Обикновено разработчиците, използващи централизираните VCS системи, избягват операциите по разделяне на клонове (*branching*), защото тези системи често не предоставят добро автоматизирано сливане на

клонове (*merging*). От друга страна децентрализираните VCS системи са по-добри в извършването на операцията *merge*, основно заради факта, че историята се пази във вид на насочен ацикличен граф. Алгоритмите за сливане се нуждаят от добра информация за историята и предшестващите версии. В това отношение насочените ациклични графове са по-подходящи в сравнение с линейното представяне на историята от централизираните VCS системи;

- Backup на хранилището, осигурен от идеята за създаване на множество инстанции – При централизираните VCS системи е нужно да се осигурява backup на сървър, на който стои централизираното хранилище, за да се избегне загуба на информация. При разпределените хранилища, дори и да не се използва специален механизъм за backup (което по принцип е задължително), може да се разчита, че множеството създадени инстанции пазят информацията от централното хранилище и тя няма да се изгуби;
- По-малки изисквания към хардуера на централното хранилище – Когато се говори за централизирана VCS система, основната идея е, че всички разработчици работят с централния сървър. Колкото повече разработчици се опитват да работят с централния сървър, толкова повече се увеличава работата на сървъра. Често дори изискванията към сървъра са изключително високи, тъй като в основата е идеята, че сървърът ще извършва много работа и към него ще се извършват много обръщения. Изискванията към централния сървър на разпределена VCS система са по-малки, защото основната работа на VCS системата по следене на версии се осъществява на локалната машина на разработчика. Централният сървър има работа само при push и pull командите към него.

5. Недостатъци на разпределените VCS системи[1]

- Липсва възможност за заключване на файлове/директории (*lock*) – Голяма част от разпределените системи не предлагат възможност за заключване. По принцип механизмът за заключване не е често използван, но за екипи, които го практикуват, е недостатък липсата му;
- Големината на локалните инстанции на хранилището – При използването на разпределени VCS разработчиците работят по своя локална инстанция на хранилището. Появява се проблем с големината на хранилището. В основата на проблема не е липсата на дисково пространство, защото цената на паметта продължава да става все по-ниска. Проблемът е в мрежовата скорост. Когато става въпрос за хранилище например от 1 TB, то в началото трябва да се създаде локална инстанция на хранилището. Това е процес, при който по мрежата се предава 1 TB от централния сървър към локалната машина на разработчика. Донякъде се намира решение на този проблем като се работи с няколко по-малки хранилища, но това може да доведе до излишно объркване на разработчиците;
- Липса на интегрираност – Когато става въпрос за работа в голям екип, то обикновено екипът не използва само обща VCS система, но и други системи, които помагат за работата на екипа. На много места кодирането е съпроводено с отстраняване и описване на бъгове, създаване на отчети, назначаване на задачи. Работата с разпределена VCS система изолира кодирането от всичко останало, което разработчиците трябва да правят. Разпределената VCS система остава отделена от останалите системи, с които разработчиците работят. Някои централизираните VCS системи (например TFS) предлагат цялостно интегрирано решение – VCS система, IDE, система за управление на проекти;
- Затруднения при унищожаване на данни – Този недостатък е другата страна на предимството с лесния backup, който се получава естествено от създаването на множество инстанции. Когато се създават множество инстанции е трудно да се изгубят данни, но когато трябва някои данни да се унищожат, то това е трудно (почти невъзможно);
- Затруднения при администрирането – При централизираните VCS системи централизираният сървър представлява мястото, където се залагат правила за сигурност, достъп, права,

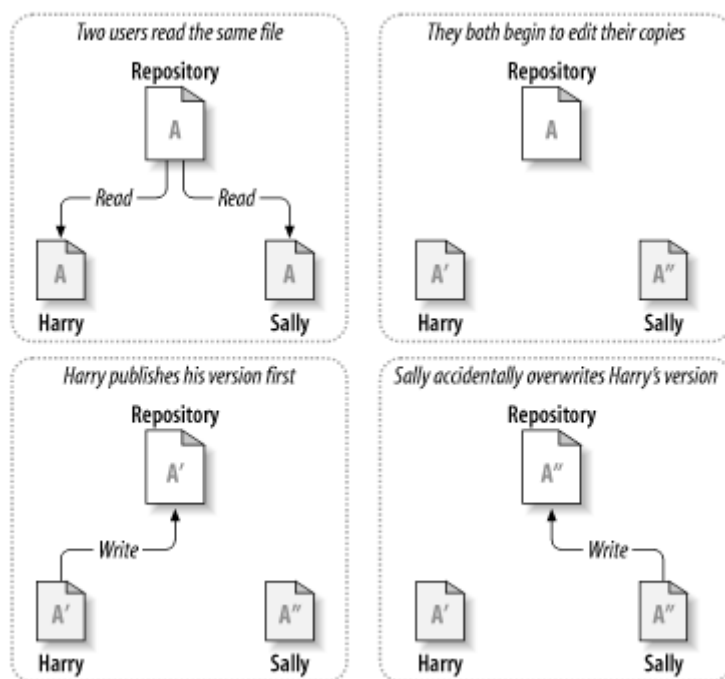
потребителски акаунти и други. Децентрализираните VCS системи са по-сложни за администриране и ограничаване на права, тъй като обикновено идентификацията на потребителите на децентрализирани системи се извършва само чрез потребителско име (което не е достатъчно);

- Няма възможност за контрол на достъпа до директории в локалното хранилище – Това произлиза от идеята, че разработчикът работи с локална инстанция на хранилището. Това означава, че той има достъп до всички файлове в хранилището и този негов достъп не може да бъде ограничен на локалната машина на разработчика;
- Лекота при използването – Въпреки, че колко лесно се използва даден софтуер, е нещо много субективно, много разработчици са съгласни, че децентрализираните VCS системи са по-трудни за използване и разбиране от централизираните;
- Липса на GUI – Много от децентрализираните VCS системи не разполагат с графичен интерфейс. На разработчиците се налага да използват команден интерфейс (command-line interface), за да работят с тях.

6. Модели, на базата на които работят системите за следене на версии[2]

VCS системите трябва да решават проблема със споделянето на информация между няколко разработчика, без разработчиците да си пречат помежду си.

Пример за показване на проблема (Фигура 6): Дадени са двама колеги Harry и Sally. И двамата решават да променят един и същи файл от едно и също хранилище в един и същи момент. Ако Harry запази своите промени в хранилището първи, тогава е възможно Sally да презапише съдържанието във файла при своя *commit*. Версията на Harry няма да се изгуби, тъй като всичко се пази в хранилището, но Sally най-вероятно няма да разбере за версията на Harry, защото тя не е виждала неговите промени. На практика промените на Harry няма да са били за дълго актуални в хранилището и може да се каже, че ще са изгубени в историята на хранилището. Възможни са два подхода за решаване на описания проблем – т. 6.1 Lock-Modify-Unlock Model, стр. 9; т. 6.2 Copy-Modify-Merge Model, стр. 10.

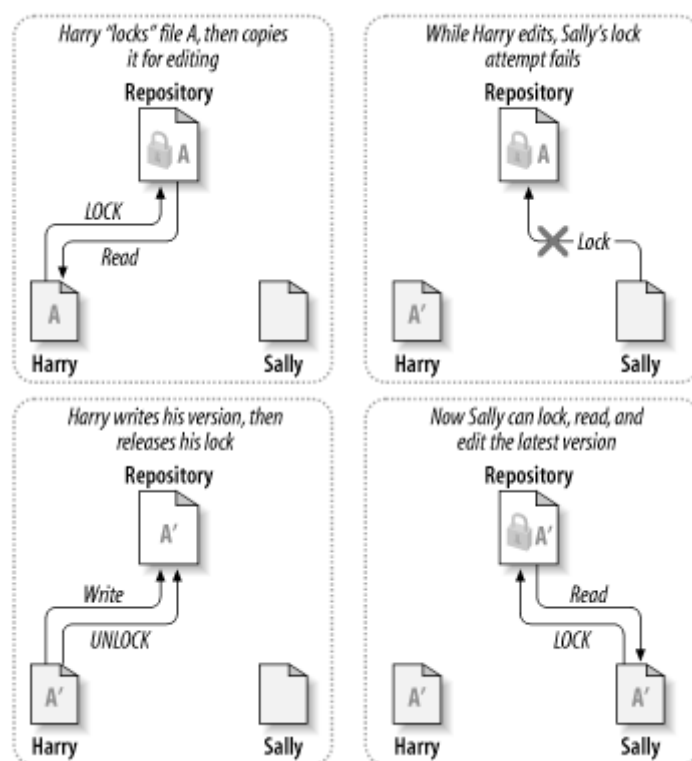


Фигура 6: VCS системи - споделяне на файлове

6.1. Lock-Modify-Unlock Model

При реализирането на този модел VCS системата позволява само на едно лице да извършва

промени по един файл в един момент. За примера (Фигура 7): Harry трябва да заключи файла, преди да започне да прави промени по него. След заключването на файла Harry може да модифицира файла, а Sally само ще може да чете съдържанието му, без да прави промени по него. Когато Harry приключи с промените си, той отключва файла и Sally може да го заключи и да започне на свой ред да модифицира файла.



Фигура 7: Lock-Modify-Unlock Model

Проблеми с този модел:

- Заклучването може да доведе до административни проблеми – Възможно е Harry да заключи файла и след това да забрави за това и да започне да върши друга работа. Междувременно Sally е възпрепятствана да продължи с работата си, защото файлът, който ѝ трябва, е заключен;
- Заклучването може да е излишно – В случай, че Harry работи по началото на файла, а Sally работи в края на файла, то те няма да си пречат. Затова в този случай заключването е излишно. Конфликтът може да бъде решен с автоматичен *merge*;
- Заклучването може да даде чувство за фалшива сигурност – Ако Harry работи по файл A и го заключва, а Sally работи по файл B и го заключва, тогава може да има проблем ако двата файла зависят един от друг. Когато и двамата разработчици приключат с работата си, тогава може да се окаже, че зависимостта между файловете вече е некоректна.

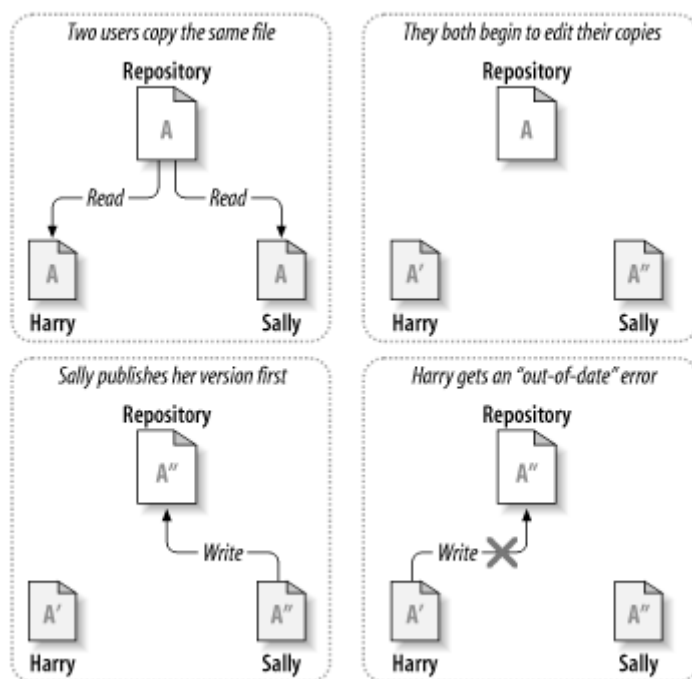
6.2. Copy-Modify-Merge Model

Този модел се явява алтернатива на заключването на файлове. В този модел всеки клиент се свързва с хранилището и си създава свое локално работно копие. Тогава потребителите могат да започнат да работят паралелно по своите локални работни копия. Когато разработчиците приключат с работата си, тогава се извършва операция по обединяване (*merge*) на работните копия, за да се формира нова версия. Обикновено се извършва автоматичен *merge*, но в някои случаи, при наличие на конфликти, се налага намесата на разработчик, който да извърши *merge* операцията.

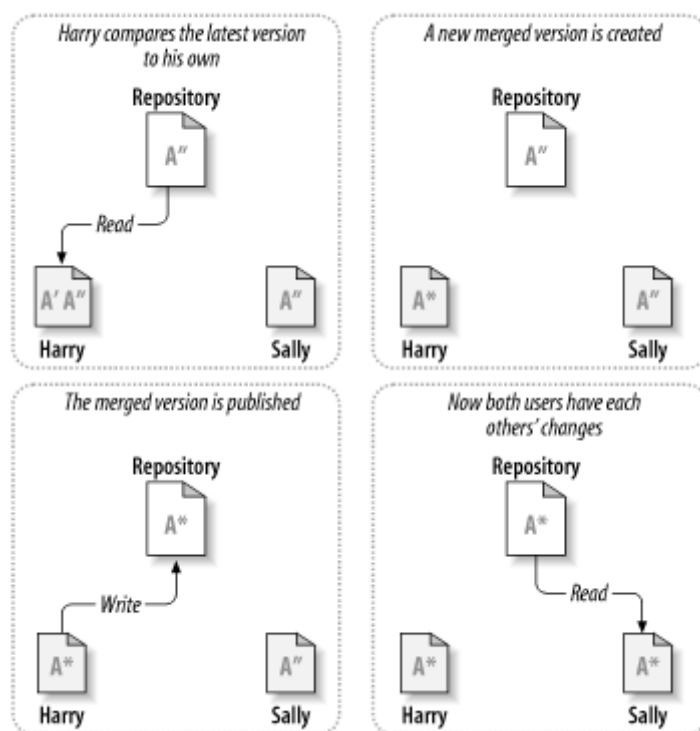
За примера (Фигура 8, Фигура 9): Harry и Sally създават работни копия на един и същи проект. Те работят конкурентно и правят промени по един и същи файл, всеки върху своето копие. Sally запазва промените си първа в хранилището. Когато Harry се опита да запази своите промени, хранилището дава

грешка от вида: Файлът, който се опитвате да *commit*-нете не е актуален. Това означава, че по някакъв начин файлът е бил променен в хранилището. За да продължи работата си, Harry трябва да съедини (*merge*) своето работно копие с данните в хранилището. Отук са възможни следващите два сюжета:

- Harry и Sally са работили по различни части от файла и затова е възможно да се направи автоматичен *merge*;
- Harry и Sally са работили по една и съща част от файла. В този случай Harry трябва да изтегли новото състояние на файла от хранилището (след промените на Sally), да го анализира, да нанесе своите промени по файла след промените на Sally, да запише новополученият файл в хранилището (*commit*).



Фигура 8: Copy-Modify-Merge Model - 1



Фигура 9: Copy-Modify-Merge Model - 2

7. Добри практики[1]

- Преди всеки *commit* проверявайте какви са промените, които сте направили (операция *diff*);
- Всеки ден проверявайте промените, които са направили другите разработчици (*diff*). Това може да подобри вашият стил на програмиране. Също така може да спомогнете за своевременното намиране на бъгове;
- За разпределени VCS системи е хубаво да се поддържат възможно най-малки хранилища;
- Групирайте *commit*-ите логически – Хубаво е даден *commit* да се свързва с решаването на дадена задача. Независимо колко файла са били променени за решаването на задачата, *commit* коментарът на всички трябва да се свързва с конкретната задача. Не е добра практика с един *commit* се свързва с решаването на няколко задачи;
- Задавайте ясен коментар за всеки от *commit*-ите. Всички VCS системи предлагат възможност *commit* операциите да са придружени от коментар. Коментарът трябва да бъде ясен и разбираем за вас и вашите колеги. Хубаво е да се включи информация не само за направените промени, но и за причината за тях;
- Съхранявайте в VCS хранилището само това, което е създадено ръчно от вас. Не е нужно да се съхраняват автоматично генерирани файлове. Съветва се да не се пазят *.exe*, *.dll* файлове. Това се свързва с нарастващия обем на хранилището. На практика за тези файлове е излишно да се пази история, след като те могат да се получат при необходимост от инструмент за автоматично генериране.

Пояснение: Разработчиците могат да съхраняват в хранилището си всякакви файлове, независимо от техния тип или начинът на тяхното създаване;

- Оставайте кода в централното хранилище работещ – Редно е когато даден разработчик изтегли проекта, той да може да го стартира, т.е. да няма грешки в проекта в централното хранилище. Не записвайте в централното хранилище некомпилиращ се код или очевидно незавършен код;

- Използвайте тагове, за да запазвате стабилното състояние на проекта си;
- Ако е необходимо ръчно сливане на файлове, преди да се направи `commit`, правете сливането внимателно. Трябва да осигурите, че при сливането няма да се изгуби съществен код;
- Не пазете код, който мислите, че ще ви потрябва, като го поставяте в коментар – Това излишно затруднява четенето на вашия код и прави кода ви изкуствено да изглежда по-дълъг. Целта на VCS системата е да пази състоянието. Веднъж, когато нещо е записано в хранилището, то при необходимост може да се възстанови;
- Проверявайте дали кодът ви се компилира успешно и дали автоматичните тестове минават след всеки `commit`.

8. Subversion (SVN)

SVN е безплатна VCS система с отворен код. Задачата ѝ е да управлява файлове и директории, и да следи за промени по тях с течение на времето. SVN позволява да се възстановяват стари версии на данните, или да се преглежда историята на промените.

В началото на 2000 г. CollabNet, Inc. започва да набира разработчици, които да започнат работа по продукт, който да замести дотогавашната масово използвана VCS система CVS. Идеята им била да създадат голям проект, в който функционалността за следене на версиите е малка част. VCS системата не е трябвало да променя фундаментално принципите, заложили в CVS, а само е трябвало да отстрани бъговете и да реши проблемите.

През 2009 CollabNet започват да работят по интегриране на SVN в Apache Software Foundation (ASF). От 2010 SVN напълно е интегриран в ASF и вече може да се открие на: <http://subversion.apache.org>.

9. Git[3]

Понастоящем Git е най-използваната модерна VCS система. Разработката на Git датира от 2005 г. И се свързва с името на Linus Torvalds (известен със създаването на ядрото на Linux ОС). Git е пример за разпределена VCS система, за разлика от CVS и SVN. При Git всеки разработчик работи върху свое работно копие на цялото хранилище.

10. Team Foundation Server (TFS) [4]

TFS се определя като Lifecycle Management система, разработена от Microsoft. Това означава, че TFS се заема с всички аспекти на софтуерната разработка – планиране, изисквания, кодиране, тестване, развиване и поддръжка. TFS съдържа в себе си Source Code Control, Bug Tracking, Project Management, Team Collaboration Platform. Продуктът е тясно свързан с Visual Studio.

11. Задачи за изпълнение

Задача 1

Изведете плюсове за използване на системи за контрол на версиите.

Задача 2

Определете за всяко поколение VCS системи кой е моделът им на работа.

Задача 3

Подгответе предоставената среда за работа с SVN и Git като използвате инструкциите в съответните документи:

- SVN – 2_version_control_systems_SVN_instructions.pdf;
- GIT – 2_version_control_systems_Git_instructions.pdf.

Пред вас е виртуална машина, създадена чрез Oracle VM, Virtual Box. ОС на виртуалната машина е Linux Mint 18.1, Serena, 32bit. Тази ОС е избрана по следните причини:

- безплатна;
- изчистена от излишни функционалности;
- опростен графичен интерфейс.

Ако за виртуалната машина са ви необходими потребител и потребителско име, то използвайте „student“.

Изключването на виртуалната машина става като изберете MENU (долния ляв ъгъл)/Quit/Shutdown.

На виртуалната машина има инсталирано IDE Eclipse – Neon.2 (Release 4.6.2). Причини за избора:

- безплатен софтуер;
- предоставя възможности за писане на различни програмни езици;
- предоставя възможности за настройка на SVN и Git клиент.

Към инсталираната версия на Eclipse е добавен допълнителния софтуер за работа с Java EE, Java Web Tools, Subversive.

Eclipse може да се стартира по някой от следните начини:

- През графичната среда:
 - От Desktop-а изберете иконката за student's Home
 - Отваря се папката eclipse
 - Клика се 2 пъти върху файл с име „eclipse“ (без разширения!)
- През командния ред:
 - Отваря се терминал
 - изпълняват се следните команди:
 - `cd /home/student/eclipse/`
 - `./eclipse`

Тази виртуална машина ще бъде използвана в упражнения 2 и 3. Ако не желаете да използвате предоставения софтуер, то можете да използвате какъвто е софтуер, който позволява работа с SVN и Git.

Може да използвате предоставената настроена виртуална машина и за изготвяне на курсовите работи.

За да се осъществява работата със системите за контрол на версиите е взето решението да се използват публични безплатни хранилища. Предимства на избора:

- не се налага настройка на сървър;
- получавате опит в използването на популярния GitHub;

- имате възможност да натрупате опит в използването на хранилища, без да ви се налага да настройвате сървър, за да изгответе курсовите си работи.

За да могат да се използват публични безплатни хранилища, ще имате нужда от допълнителни стъпки при настройката на средата, които са описани в допълнителните файлове.

Използвана литература

- 1: Eric Sink, Version Control By Example, 2011, http://ericsink.com/vcbe/vcbe_a4_lo.pdf
- 2: Ben Collins-Sussman, Brian W. Fitzpatrick, and C. Michael Pilato, Version Control with Subversion, 2011, <http://svnbook.red-bean.com/en/1.0/ch02s02.html>
- 3: Atlassian Team, What is Git, , <https://www.atlassian.com/git/tutorials/what-is-git>
- 4: Team Foundation ServerHans-Petter Halvorsen, Team Foundation Server, , http://home.hit.no/~hansha/documents/software/software_development/topics/resources/TFS%20Overview.pdf